

MOVIELENS HYBRID RECOMMENDATION SYSTEM

Tài liệu phân tích kiến trúc chi tiết

History-Based Switching with Optional Fusion

Mục tiêu tài liệu: Giúp người đọc hiểu từ flow tổng quát đến từng block, từng mũi tên, dữ liệu đi đâu, vì sao đi như vậy, khi nào component chạy, input/output là gì và mối quan hệ giữa Batch, Streaming, Recommendation, Serving và Model Refresh.

Ngôn ngữ: Tiếng Việt - giữ nguyên các technical keywords quan trọng bằng English

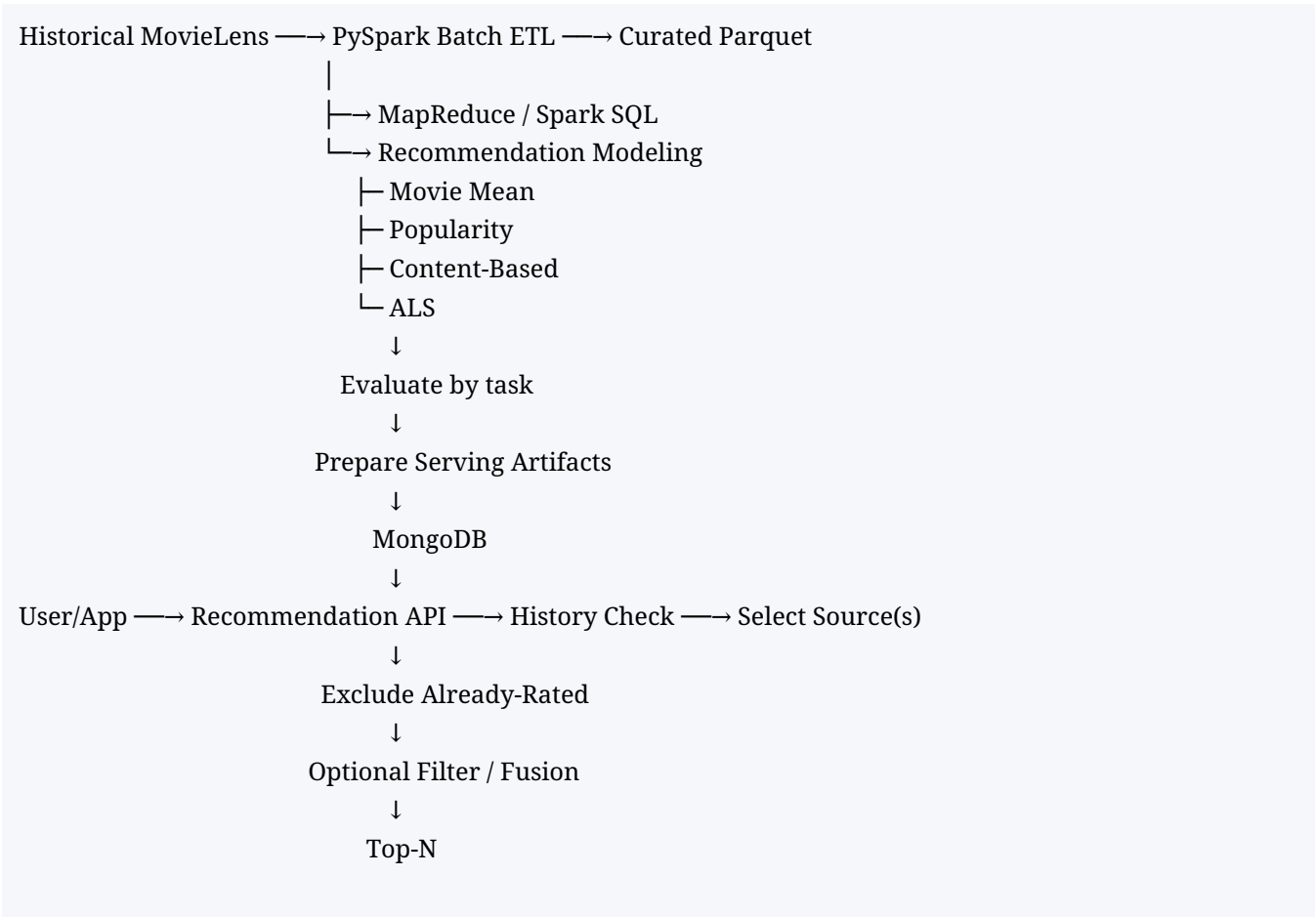
1. Cách đọc kiến trúc này

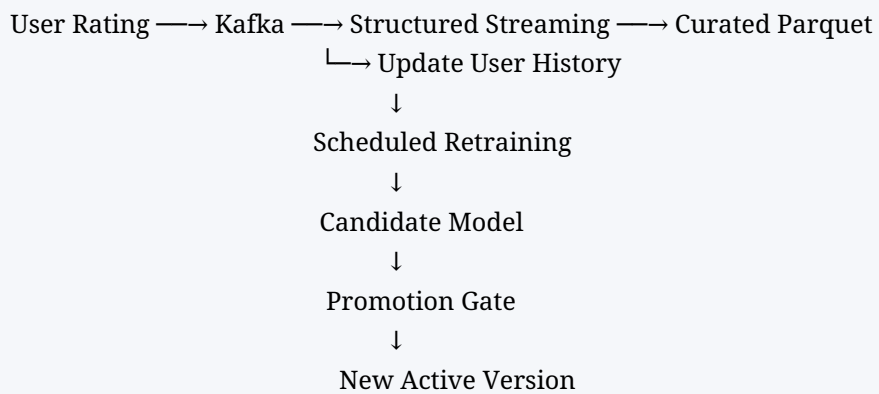
Kiến trúc không phải là một chuỗi duy nhất từ trái sang phải. Nó gồm ba vòng đời chạy song song nhưng liên kết với nhau:

Vòng đời	Mục đích	Bắt đầu từ	Kết thúc ở
Data lifecycle	Nhận dữ liệu, kiểm tra, chuẩn hóa và lưu dữ liệu dùng chung.	MovieLens CSV hoặc User Rating event	Curated Parquet + User History
Recommendation lifecycle	Huấn luyện/evaluate các strategy, tạo serving artifacts và trả Top-N.	Curated Parquet + User request	Top-N Response
Model lifecycle	Tích lũy feedback, retrain candidate model, validate rồi promote version mới.	Accumulated curated data	MongoDB active version mới

Tư duy cốt lõi: Batch xử lý lịch sử lớn; Streaming tiếp nhận rating mới; Offline Modeling tạo artifacts; Online Serving chỉ lookup/merge nhẹ; Feedback không retrain ALS ngay lập tức mà tích lũy dữ liệu rồi retrain định kỳ.

1.1 Flow tổng quát ở mức 30 giây

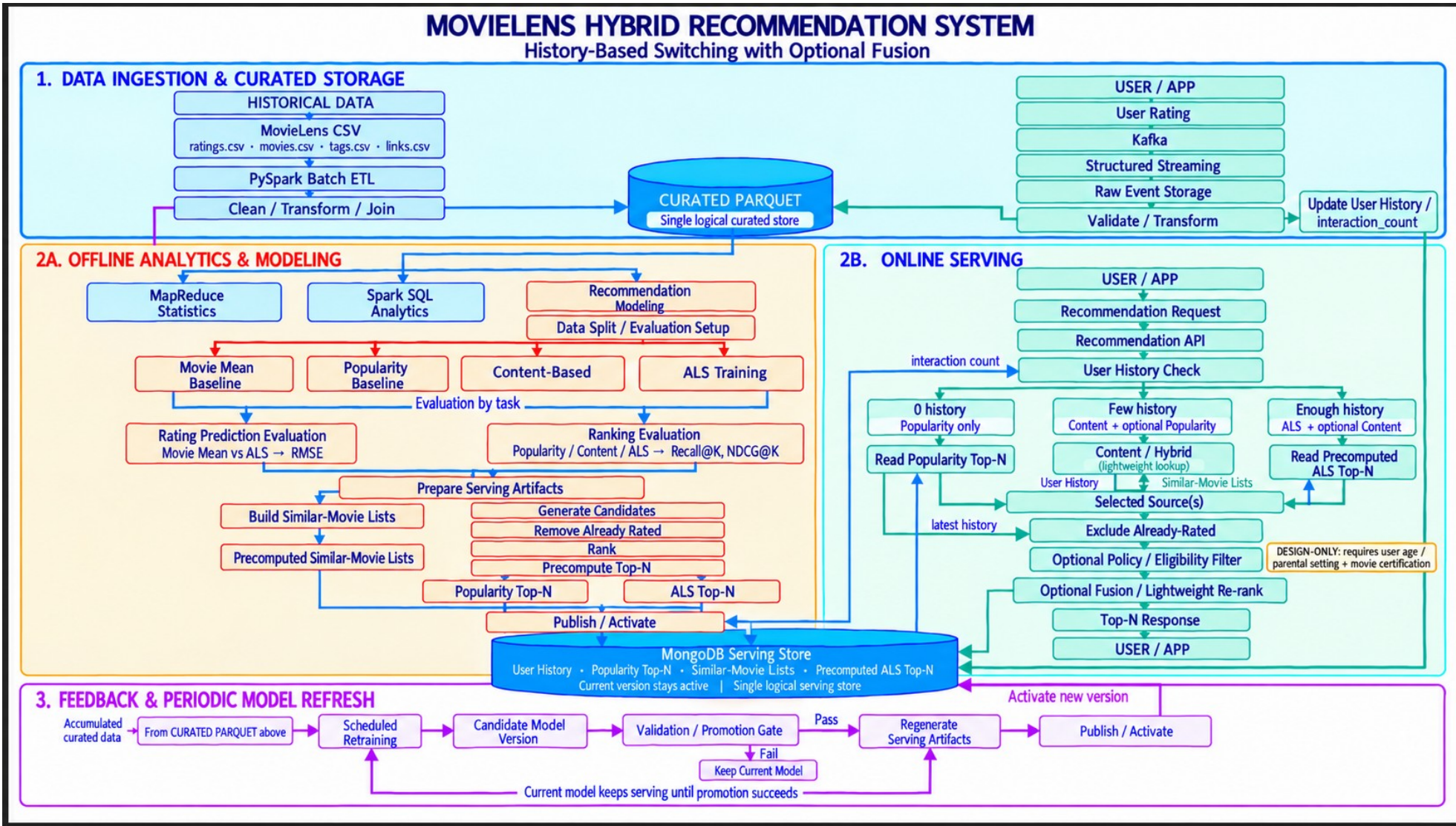




1.2 Ba câu hỏi kiến trúc phải trả lời

- Dữ liệu mới đi vào hệ thống bằng đường nào và được lưu ở đâu?
- Khi một user yêu cầu recommendation, hệ thống chọn strategy nào và lấy dữ liệu từ đâu?
- Khi rating mới tích lũy, lúc nào model được retrain và làm sao version mới trở thành active version?

2. Kiến trúc tổng thể - bản tham chiếu



Hình 1. MovieLens Hybrid Recommendation System - History-Based Switching with Optional Fusion

3. Layer 1 - Data Ingestion & Curated Storage

Layer này có hai đường vào độc lập: Historical Batch và Live Rating Events. Hai đường cùng chuẩn hóa về một nguồn dữ liệu chung là `Curated Parquet`; riêng live events còn cập nhật `User History / interaction\_count` để Online Serving nhìn thấy thay đổi mới.

3.1 Historical path

Block	Vai trò	Input	Output
HISTORICAL DATA	Khái niệm nguồn dữ liệu lịch sử, không phải một database.	Không có	MovieLens CSV
MovieLens CSV	`ratings.csv`, `movies.csv`, `tags.csv`, `links.csv`.	Các file raw	Input cho PySpark Batch ETL
PySpark Batch ETL	Đọc file bằng schema rõ ràng và tạo distributed DataFrames.	CSV	DataFrames có typed columns
Clean / Transform / Join	Kiểm tra null/invalid/duplicate, cast type, đổi timestamp, join metadata theo `movieId`.	DataFrames	Clean/enriched DataFrames
CURATED PARQUET	Single logical curated store cho Analytics + Modeling + Retraining.	Clean DataFrames	Parquet datasets

Arrow H1: Historical Data → MovieLens CSV

Ý nghĩa: chọn MovieLens 32M làm nguồn historical benchmark. Đây là bước logical source selection, không có computation.

Arrow H2: MovieLens CSV → PySpark Batch ETL

Payload: các dòng CSV raw. PySpark chịu trách nhiệm đọc phân tán, áp `schema`, tạo DataFrames và chuẩn bị cho các transformations tiếp theo.

Arrow H3: PySpark Batch ETL → Clean / Transform / Join

Đây là data-quality boundary. Dữ liệu đã đọc nhưng chưa chắc đã đủ sạch để dùng cho model/analytics.

Arrow H4: Clean / Transform / Join → Curated Parquet

Chỉ dữ liệu đã qua validation/standardization mới được publish vào curated layer. Downstream không nên quay lại raw CSV để tự làm sạch lại.

3.2 Streaming path

Block	Vai trò	Input	Output
USER / APP	Nguồn phát sinh event thật hoặc simulator trong demo.	User interaction	User Rating
User Rating	Event explicit feedback: `userId`, `movieId`, `rating`, `timestamp`.	Action từ user	Kafka message
Kafka	Event transport / durable	Rating event	Kafka topic records

Block	Vai trò	Input	Output
	log; decouple producer và consumer.		
Structured Streaming	Đọc Kafka liên tục bằng Spark Structured Streaming.	Kafka records	Streaming DataFrame
Raw Event Storage	Lưu event gốc/replayable trước khi curate.	Streaming record	Raw event copy
Validate / Transform	Parse schema, kiểm tra rating range, type, duplicate/event validity, normalize timestamp.	Raw events	Validated events
Update User History / interaction_count	Cập nhật online state để History Check nhìn thấy feedback mới mà không cần đợi retraining.	Validated rating event	Latest history/count

Arrow S1: User/App → User Rating

Một action của user được chuyển thành một event chuẩn hóa. Trong project, đây có thể là simulated event vì MovieLens 32M là historical dataset.

Arrow S2: User Rating → Kafka

Producer ghi event vào topic. Kafka không train model; nó chỉ vận chuyển và giữ event để consumer xử lý.

Arrow S3: Kafka → Structured Streaming

Spark đọc stream theo offsets/micro-batch hoặc continuous-style execution. Mục tiêu là xử lý events mới incremental thay vì đọc lại toàn bộ dataset.

Arrow S4: Structured Streaming → Raw Event Storage

Raw event được lưu để audit/replay. Nếu transform downstream thay đổi, event gốc vẫn có thể xử lý lại.

Arrow S5: Raw Event Storage → Validate / Transform

Event gốc phải được kiểm tra trước khi ảnh hưởng tới curated data hoặc online state.

Arrow S6a: Validate / Transform → Curated Parquet

Validated rating được append vào curated dataset để lần retraining sau sử dụng.

Arrow S6b: Validate / Transform → Update User History / interaction_count

Đây là đường latency thấp phục vụ routing online. Nó chỉ cập nhật state phục vụ quyết định 0/Few/Enough history; không đồng nghĩa ALS đã học rating mới.

Arrow S7: Update User History → MongoDB Serving Store

Online serving state phải được ghi về Serving Store để request tiếp theo nhìn thấy lịch sử mới nhất.

Điểm rất quan trọng: `Streaming ingestion` và `ALS retraining` là hai việc khác nhau. Rating mới có thể xuất hiện trong User History ngay, nhưng chỉ ảnh hưởng latent factors của ALS sau lần Scheduled Retraining kế tiếp.

4. Layer 2A - Offline Analytics & Modeling

Layer này tiêu thụ `Curated Parquet`. Ba workload lớn chạy song song về mặt logic: MapReduce Statistics, Spark SQL Analytics và Recommendation Modeling. Chúng dùng cùng nguồn curated nhưng không phụ thuộc nối tiếp nhau.

4.1 Curated Parquet → ba workload

Nhánh	Mục tiêu	Kết quả
MapReduce Statistics	Chứng minh mapper-shuffle-reducer; thống kê mergeable như `(sumRating, count)` theo `movieId`.	Movie-level statistics + cross-check với Spark
Spark SQL Analytics	Phân tích popularity, genre, temporal activity, execution plan.	Analytics tables / reports
Recommendation Modeling	Xây và evaluate các recommendation strategies.	Serving artifacts

4.2 Data Split / Evaluation Setup

Block này định nghĩa cách tách dữ liệu và ground truth trước khi build/evaluate. Với project course có thể dùng fixed-seed split; nếu muốn gần production hơn có thể dùng temporal holdout.

- `Train`: dùng để fit ALS và tính statistics/features mà không nhìn test labels.
- `Validation`: dùng để chọn hyperparameters / threshold nếu có.
- `Test`: chỉ dùng để báo cáo metric cuối.
- Với Top-N metrics phải định nghĩa `relevant item`, ví dụ `rating >= 4.0` trong holdout.

4.3 Movie Mean Baseline

Baseline cho rating prediction. Với mỗi movie, dự đoán bằng mean rating của movie trên train set; movie unseen có thể fallback về global mean.

```
prediction(movie_i) = mean_train_rating(movie_i)
RMSE_baseline = sqrt(mean((rating_true - prediction)^2))
```

Movie Mean không dùng để online Top-N serving trong kiến trúc hiện tại; vai trò chính là mốc so sánh với ALS.

4.4 Popularity Baseline

Baseline cho Top-N ranking và fallback cho user 0-history. Có thể dùng `rating\_count`, `average\_rating` với `minimum support`, hoặc weighted score. Artifact cuối là `Popularity Top-N`.

4.5 Content-Based

Content-Based dùng movie metadata như `genres` và `tags` để xây representation của movie. Thay vì online quét toàn bộ catalog, kiến trúc precompute `Similar-Movie Lists` offline.

```
Movie metadata
→ feature representation
→ similarity(movie_i, movie_j)
→ Top similar movies per movie
```

→ Precomputed Similar-Movie Lists

Khi user có few history, hệ thống dùng những movie user thích gần đây + Similar-Movie Lists để tạo candidate set.

4.6 ALS Training

ALS là primary collaborative model. Input cốt lõi là `userId`, `movieId`, `rating`. ALS học `user latent factors` và `item latent factors`; predicted preference được tính bằng dot product giữa hai vectors.

Input interactions: (userId, movieId, rating)

ALS learns:

U = user latent-factor matrix

V = movie latent-factor matrix

Predicted score:

$$\hat{r}(u,i) = U[u] \cdot V[i]$$

Training alternates:

fix V → solve U

fix U → solve V

repeat

Spark ALS có `coldStartStrategy` để xử lý unknown ids trong prediction/evaluation, nhưng business cold start cho user mới vẫn cần routing/fallback riêng.

4.7 Evaluation by task

Task	So sánh	Metric	Ý nghĩa
Rating Prediction	Movie Mean vs ALS	RMSE	Prediction gần rating thật đến mức nào.
Top-N Ranking	Popularity / Content / ALS	Recall@K, NDCG@K	Relevant items có xuất hiện và nằm cao trong Top-K hay không.

4.8 Prepare Serving Artifacts

Sau evaluation, hệ thống không đưa training object thô trực tiếp ra Online Serving. Nó tạo các artifacts có thể lookup nhanh:

Artifact	Được tạo từ	Dùng khi online
Popularity Top-N	Popularity Baseline	0-history fallback; optional fusion
Precomputed Similar-Movie Lists	Content-Based	Few-history; optional content enrichment
Precomputed ALS Top-N	ALS	Enough-history users

4.9 Candidate generation pipeline cho ALS

1. Generate Candidates: ALS sinh candidate movies cho user.

2. Remove Already Rated: loại movie đã có trong training history tại thời điểm precompute.
3. Rank: sort theo ALS score.
4. Precompute Top-N: giữ N item đứng đầu để serving nhanh.

Online vẫn có thêm 'Exclude Already-Rated' vì user có thể rating thêm movie mới sau lần precompute cuối.

4.10 Publish / Activate → MongoDB

Artifacts được publish vào 'MongoDB Serving Store'. MongoDB là read-optimized serving layer trong kiến trúc này; không phải nơi train ALS.

5. MongoDB Serving Store - hợp đồng dữ liệu giữa Offline và Online

MongoDB là điểm giao giữa Offline Modeling và Online Serving. Offline ghi artifacts; Online đọc theo access pattern cụ thể.

Dữ liệu	Key gợi ý	Ví dụ use case
User History	`userId`	History Check, Content/Hybrid, Exclude Already-Rated
Popularity Top-N	`segment/global`	0-history fallback
Similar-Movie Lists	`movieId`	Từ movie user thích → lấy similar candidates
Precomputed ALS Top-N	`userId`	Enough-history recommendation
Version metadata	`modelVersion`, `generatedAt`	Theo dõi active version, rollback/audit

Data modeling: Top-N lists nên là bounded arrays/documents. Không lưu lịch sử hoặc recommendation arrays tăng vô hạn; chỉ giữ phần cần phục vụ.

6. Layer 2B - Online Serving: một request đi qua hệ thống như thế nào

6.1 Flow tổng quát

```

USER / APP
→ Recommendation Request
→ Recommendation API
→ User History Check
→ route by interaction_count
→ Selected Source(s)
→ Exclude Already-Rated
→ Optional Policy / Eligibility Filter
→ Optional Fusion / Lightweight Re-rank
→ Top-N Response
→ USER / APP
  
```

6.2 Recommendation Request → Recommendation API

Request mang tối thiểu `userId`; có thể có `K`, context hoặc policy flags. API không nên tự train model. Nhiệm vụ chính là orchestration và lookup.

6.3 User History Check

API đọc `interaction\_count` và latest `User History` từ MongoDB. Đây là quyết định routing, không phải model prediction.

Tier	Điều kiện concept	Strategy chính	Lý do
0 history	Không có usable interaction	Popularity only	Không có user factor/profile đáng tin cậy
Few history	Có ít interactions	Content + optional Popularity	Content signal hữu ích hơn collaborative factor chưa ổn định
Enough history	Đủ interaction theo threshold	ALS + optional Content	ALS đã có personalization signal tốt hơn

Threshold `Few/Enough` là design parameter của prototype; không phải hằng số khoa học phổ quát.

6.4 0 history → Read Popularity Top-N

Hệ thống đọc list đã precompute trong MongoDB. Không chạy ALS vì chưa có `user factor` hữu ích cho user mới.

6.5 Few history → Content / Hybrid

Inputs online gồm `User History` + `Similar-Movie Lists`. Hệ thống lấy các movie user đã thích/rating cao, truy vấn similar lists, hợp nhất candidates; có thể thêm Popularity như fallback/regularizer.

```
Few-history user
→ select positive/recent history movies
→ lookup Similar-Movie Lists
→ union candidates
→ optional add Popularity candidates
→ de-duplicate
→ Selected Source(s)
```

6.6 Enough history → Read Precomputed ALS Top-N

Online không chạy Spark ALS cho mỗi request. Nó đọc Top-N đã precompute bằng `userId`, giúp latency ổn định.

6.7 Selected Source(s)

Đây là abstraction hợp nhất output từ strategy được router chọn. Nếu chỉ có một source thì pass-through; nếu có optional fusion thì nhiều source được hợp nhất.

6.8 Exclude Already-Rated

Online exclusion dùng `latest history`, khác với offline removal. Nó xử lý trường hợp user vừa rate một movie sau lần precompute gần nhất.

6.9 Optional Policy / Eligibility Filter

Design-only extension. Ví dụ content restriction theo tuổi chỉ khả thi nếu hệ thống có user age/parental setting và movie certification từ nguồn phù hợp. MovieLens 32M không có demographic age.

Không giao policy cho ALS: ALS tối ưu relevance từ interaction patterns; eligibility/safety là business rule và nên nằm ở filter layer.

6.10 Optional Fusion / Lightweight Re-rank

Nếu router chọn nhiều nguồn, online có thể normalize scores và fusion/rerank nhẹ. Ví dụ few-history = Content + optional Popularity; enough-history = ALS + optional Content.

Example concept:

$$\text{final_score} = \alpha * \text{normalized_primary_score} + \beta * \text{normalized_secondary_score}$$

α, β are design/tuning parameters, not fixed truths.

Nếu chỉ có một source, block này có thể chỉ làm pass-through + tie-break/re-rank nhẹ.

6.11 Top-N Response

API trả N items cuối cùng, kèm metadata cần hiển thị như `movieId`, `title`, `genres`, `rank`, `score`, `strategy`, `modelVersion` nếu cần audit.

7. Layer 3 - Feedback & Periodic Model Refresh

Layer này giải quyết câu hỏi: rating mới đã được ingest rồi, khi nào ALS và serving artifacts được cập nhật?

```
Accumulated curated data
→ Scheduled Retraining
→ Candidate Model Version
→ Validation / Promotion Gate
  |— Fail → Keep Current Model
  |— Pass → Regenerate Serving Artifacts
        → Publish / Activate
        → MongoDB active version
```

7.1 Accumulated curated data

Nguồn là `Curated Parquet` đã chứa historical data + validated new rating events.

7.2 Scheduled Retraining

Trigger theo lịch hoặc volume threshold. ALS không retrain trên từng Kafka event. Điều này giảm compute cost và tránh model churn.

7.3 Candidate Model Version

Kết quả retrain là candidate version, chưa được phép serve ngay. Cần lưu config, seed, metrics và artifact metadata để reproducible.

7.4 Validation / Promotion Gate

Gate kiểm tra model mới trước khi promote. Có thể gồm:

- RMSE không tệ hơn baseline/active model quá mức cho phép.
- Recall@K / NDCG@K đạt acceptance criteria.
- Artifact integrity: đủ users/items, không duplicate, Top-N hợp lệ.
- Latency/storage sanity checks nếu có.
- Schema/version compatibility.

7.5 Fail → Keep Current Model

Nếu candidate không đạt, serving store và active metadata không đổi. Hệ thống tiếp tục phục vụ version cũ.

7.6 Pass → Regenerate Serving Artifacts

Candidate model đạt gate thì tạo lại Precomputed ALS Top-N và các artifacts cần refresh. Không nhất thiết phải rebuild mọi artifact nếu chúng không phụ thuộc candidate.

7.7 Publish / Activate

Sau khi artifacts hoàn tất, hệ thống atomically/transactionally theo khả năng triển khai để đổi active version. Mục tiêu là tránh trạng thái nửa cũ nửa mới.

8. Danh mục mũi tên - ý nghĩa từng connection

ID	Connection	Ý nghĩa
A01	MovieLens CSV → PySpark Batch ETL	Đọc raw historical data thành distributed DataFrames.
A02	PySpark Batch ETL → Clean/Transform/Join	Chuyển từ ingestion sang data preparation.
A03	Clean/Transform/Join → Curated Parquet	Publish dữ liệu đã chuẩn hóa.
A04	User Rating → Kafka	Producer phát event.
A05	Kafka → Structured Streaming	Consumer đọc event stream.
A06	Structured Streaming → Raw Event Storage	Lưu raw/replayable event.
A07	Raw Event Storage → Validate/Transform	Kiểm tra trước khi curate.
A08	Validate/Transform → Curated Parquet	Append validated event cho retraining/analytics.
A09	Validate/Transform → Update User History	Cập nhật online state nhanh.
A10	Update User History → MongoDB	Để request tiếp theo thấy interaction mới.
A11	Curated Parquet → MapReduce	Statistics workload.
A12	Curated Parquet → Spark SQL	Analytics workload.
A13	Curated Parquet → Recommendation Modeling	Training/evaluation workload.
A14	Data Split → Movie	Cùng evaluation setup, khác algorithm.

ID	Connection	Ý nghĩa
	Mean/Popularity/Content/ALS	
A15	Movie Mean + ALS → Rating Prediction Evaluation	RMSE comparison.
A16	Popularity + Content + ALS → Ranking Evaluation	Recall@K / NDCG@K.
A17	Evaluation → Prepare Serving Artifacts	Chỉ artifacts đạt criteria mới chuẩn bị serve.
A18	Content-Based → Similar-Movie Lists	Precompute content neighbors.
A19	ALS → Generate Candidates → ALS Top-N	Precompute personalized list.
A20	Popularity → Popularity Top-N	Precompute fallback list.
A21	Serving Artifacts → Publish/Activate → MongoDB	Offline-to-online handoff.
A22	MongoDB interaction_count → User History Check	Routing input.
A23	0 history → Popularity Top-N	Fallback route.
A24	Few history → Content/Hybrid	Sparse-history route.
A25	Enough history → ALS Top-N	Collaborative route.
A26	User History + Similar-Movie Lists → Content/Hybrid	Online content candidate lookup.
A27	Selected Source(s) → Exclude Already-Rated	Use latest history to remove stale items.
A28	Exclude → Optional Policy Filter	Apply eligibility/business rules.
A29	Filter → Optional Fusion/Re-rank	Combine/rerank only when needed.
A30	Re-rank → Top-N Response	Return final ordered list.
A31	Curated Parquet → Scheduled Retraining	Model refresh input.
A32	Retraining → Candidate Model	Create non-active version.
A33	Candidate → Promotion Gate	Quality/safety decision.
A34	Fail → Keep Current Model	No production change.
A35	Pass → Regenerate Artifacts	Prepare new serving data.
A36	Publish/Activate → MongoDB	Switch active artifacts/version.

9. Walkthrough theo 4 tình huống thực tế

9.1 User mới chưa có rating

- User gửi Recommendation Request.
- API đọc interaction_count = 0.
- Router chọn Popularity only.
- Đọc Popularity Top-N từ MongoDB.
- Exclude Already-Rated gần như không loại gì.
- Optional policy filter nếu có.

11. Trả Top-N.

9.2 User có 3 rating

12. API đọc Few history.
13. Lấy latest User History.
14. Từ movie user thích, lookup Similar-Movie Lists.
15. Có thể thêm Popularity candidates.
16. De-duplicate → Exclude Already-Rated.
17. Optional fusion/lightweight re-rank.
18. Trả Top-N.

9.3 User có đủ history

19. API route sang Enough history.
20. Đọc Precomputed ALS Top-N.
21. Optional Content source nếu design bật fusion.
22. Dùng latest history loại phim vừa rating sau lần precompute.
23. Re-rank nhẹ nếu cần.
24. Trả Top-N.

9.4 User vừa rating thêm một phim

25. Rating event vào Kafka.
26. Structured Streaming xử lý event.
27. Validated event append vào Curated Parquet.
28. User History / interaction_count trong MongoDB được cập nhật ngay.
29. Request kế tiếp sử dụng history mới để exclude/routing.
30. ALS Top-N vẫn từ active model cũ.
31. Đến Scheduled Retraining, rating mới mới ảnh hưởng latent factors và ALS Top-N mới.

10. Giả định, giới hạn và ranh giới khoa học

Điểm	Phải hiểu đúng
MovieLens 32M	Là historical dataset; mọi selected user có ít nhất 20 ratings. `0/Few history` phải simulated bằng masking/truncation khi evaluate cold start.
Age restriction	MovieLens không có age/demographics; policy filter chỉ là design-only nếu chưa enrich external data.
ALS coldStartStrategy	Giúp xử lý unknown ids trong prediction/evaluation; không thay thế business fallback strategy.
Streaming	Giải quyết event ingestion / online state update; không có nghĩa ALS online-learning theo từng event.
Hybrid	Hệ thống dùng history-based switching; fusion là optional. Không phải mọi request đều blend nhiều models.
RMSE	Chỉ dùng cho rating prediction path.
Recall@K/NDCG@K	Dùng cho ranking; cần định nghĩa relevant ground truth rõ ràng.

Điểm	Phải hiểu đúng
MongoDB	Serving store/read model; không phải training store.

11. Mapping từ architecture sang implementation

Architecture Block	Implementation gợi ý	Output kiểm chứng
PySpark Batch ETL	<code>`spark.read.schema(...).csv(...)`</code>	schema + row counts
Clean/Transform/Join	DataFrame transformations / joins	clean counts + unmatched ids
Curated Parquet	<code>`write.mode(...).parquet(...)`</code>	Parquet folders + read-back
MapReduce	mapper/reducer <code>`(sum,count)`</code>	compare Spark groupBy
Spark SQL	temp views / SQL queries / <code>`explain('formatted')`</code>	analytics + plan
ALS	<code>`pyspark.ml.recommendation.ALS`</code>	model + predictions
Rating evaluation	<code>`RegressionEvaluator(metricName='rmse')`</code>	RMSE baseline vs ALS
Ranking evaluation	Top-K ground truth + <code>RankingEvaluator/custom</code>	Recall@K/NDCG@K
Kafka + Structured Streaming	<code>`readStream.format('kafka')`</code>	event ingestion + checkpoint
MongoDB	write/read serving documents	find by <code>`userId`</code>
Promotion Gate	script/config thresholds	pass/fail + active version

12. Tài liệu kỹ thuật tham chiếu

- GroupLens - MovieLens 32M README: <https://files.grouplens.org/datasets/movielens/ml-32m-README.html>
- Apache Spark - Structured Streaming: <https://spark.apache.org/streaming/>
- Apache Spark - Structured Streaming + Kafka Integration: <https://spark.apache.org/docs/3.5.6/structured-streaming-kafka-integration.html>
- Apache Spark - ALS / Collaborative Filtering documentation: <https://spark.apache.org/docs/latest/api/scala/org/apache/spark/ml/recommendation/ALS.html>
- MongoDB - Embedded Data Modeling: <https://www.mongodb.com/docs/manual/data-modeling/embedding/>
- MongoDB - Avoid Unbounded Arrays: <https://www.mongodb.com/docs/v8.0/data-modeling/design-antipatterns/unbounded-arrays/>

Kết luận: Kiến trúc được hiểu đúng khi tách rõ bốn khái niệm: dữ liệu được ingest liên tục, online state được cập nhật nhanh, recommendation serving chủ yếu đọc artifacts đã chuẩn bị, còn model learning/retraining diễn ra định kỳ trong offline lifecycle.